PIN Generation Using String Manipulation

Write a Java program to generate a PIN based on two names, a six-digit PIN, and a position value (N).

The program should accept the first name, last name, a six-digit positive PIN, and an integer (N) as input. It must identify the longer and shorter names based on their lengths. If both names have the same length, the name that comes later in lexicographical order should be considered the longer name.

Construct a new string by concatenating:

1. The first character of the longer name.
2. The complete shorter name.
3. The (N)th digit of the PIN from the left.
4. The (N)th digit of the PIN from the right.

Finally, toggle the case of every alphabetic character in the constructed string: convert uppercase letters to lowercase and lowercase letters to uppercase. Digits must remain unchanged.

Input Validation Requirements:

- Both names must not be null or empty after trimming leading and trailing whitespace.
- The PIN must be a six-digit positive integer.
- The value of (N) must be between 1 and 6, inclusive.
- If any input is invalid, display an appropriate error message.

Sample Input:

```
First Name: Rajiv
Last Name: Roy
PIN: 560037
N: 5
```

Sample Output:

```
Generated PIN: rROy03
```

Explanation:

- The longer name is `Rajiv`, and the shorter name is `Roy`.
- The first character of the longer name is `R`.
- The 5th digit from the left in `560037` is `3`.
- The 5th digit from the right in `560037` is `0`.
- The constructed string is `RRoy30`.
- After toggling the case, the final generated PIN is `rROy30`.

Note: The correct sample output is `rROy30`, based on the specified rules.

Below are the test cases for the PIN Generation Using String Manipulation problem, covering valid inputs, name-length comparisons, case toggling, and input validation.

Test cases

TC ID	First Name	Last Name	PIN	N	Expected Output
TC01	Rajiv	Roy	560037	5	`rROy30`
TC02	Amit	Sharma	123456	2	`SaMIT25`
TC03	Rahul	Verma	987654	3	`vRAHUL76`
TC04	Anil	Sunil	123456	1	`sANIL16`
TC05	Amit	Ravi	456789	6	`rAMIT96`
TC06	Raj	Kumar	234567	4	`kRAJ56`
TC07	Rajiv	Roy	12345	3	Error: PIN must be a six-digit positive number
TC08	Rajiv	Roy	1234567	3	Error: PIN must be a six-digit positive number
TC09	Rajiv	Roy	560037	0	Error: N must be between 1 and 6
TC10	Rajiv	Roy	560037	7	Error: N must be between 1 and 6
TC11	Empty string	Roy	560037	3	Error: First name cannot be null or empty
TC12	Rajiv	Empty string	560037	3	Error: Last name cannot be null or empty
TC13	`null`	Roy	560037	3	Error: First name cannot be null or empty
TC14	Rajiv	Roy	-12345	3	Error: PIN must be a six-digit positive number
TC15	A	B	100001	1	`bA10`

Additional test cases for name comparison

TC ID	First Name	Last Name	PIN	N	Expected behavior
TC16	Amit	Ravi	123456	3	Equal-length names; `Ravi` is selected as the longer name
TC17	Ravi	Amit	123456	3	Equal-length names; `Ravi` is selected as the longer name
TC18	`Rajiv`	`Roy`	560037	5	Trim whitespace and generate the PIN as in TC01

Important corrections for your Java implementation:

1. Your `generatePin()` method should validate the inputs itself, or its callers must always validate them before calling it. Currently, direct calls to `generatePin()` can bypass validation.
2. The current `validateNumber()` method checks only the range of `N`; it does not validate the PIN. Call `validatePin(pin)` before checking `N`.
3. For TC15, the correct output under your stated rules is `bA10`: the longer name is `B` by the equal-length lexicographical comparison, and the shorter name is `A`. The selected PIN digits are both `1`, so the constructed string `BA11` toggles to `ba11`. Therefore, the expected output should be `ba11`.